# GPU-Accelerated 3D Gaussian Splatting SLAM

15-418/15-618 Final Project Proposal

Nick Yaeger (nyaeger) Oliver Yun (oyun)

November 9, 2026

---

## Project Website

https://oliveryun11.github.io/15418-gaussian-slam/

## 1 Project Summary

We accelerate 3D Gaussian splatting simultaneous localization and mapping (SLAM), which builds a 3D scene from RGB-D camera input while estimating the camera's position, via algorithmic optimizations and GPU parallelization, running on NVIDIA A100 GPUs on NCSA's Delta cluster. We implement sparse pixel sampling and pixel-based CUDA kernels for rendering and gradient computation in both pose estimation and mapping, and compare against a tile-based baseline. We measure how each optimization affects speed and accuracy to determine how close GPU-level optimization can bring Gaussian splatting SLAM to real-time performance.

## 2 Background

Simultaneous localization and mapping (SLAM) is the task of estimating the pose and position of a vision sensor while simultaneously building a map of the environment. SLAM has important applications in robotics, autonomous driving, and augmented reality, where a system must understand its surroundings and its own motion in real time. Recent work such as SplaTAM [2] represents the map as a set of 3D Gaussians, defined by position, size/stretch, color, and opacity. Given an RGB-D camera, which captures a color image and per-pixel depth, SplaTAM can produce high-quality, photo-realistic maps. However, SplaTAM is slow, running at under 1 frame per second on a modern GPU.

At a high level, the algorithm works by repeatedly updating the camera pose and the map of Gaussians for each frame. For each frame, the algorithm runs in three steps. First, a new camera pose is estimated by repeatedly rendering the map from the current pose estimate, comparing the result with the new frame, and updating the pose using gradient descent. Then, new Gaussians are added in sections not yet covered by the map. Finally, Gaussians are refined by repeatedly rendering from known camera poses, comparing render results with real frames, and updating the Gaussians' parameters using gradient descent.

**Algorithm 1** Gaussian Splatting SLAM

```
Initialize empty Gaussian map G.
for all Frames F_i, i ∈ {1, ..., T} do
    Guess pose P_i from previous poses P_{i−1}, P_{i−2}.
    for N_track iterations do
        Sample a subset S of pixels from F_i.
        Render the scene R using pose P_i, pixels S.
        Compute the error E between scene R and frame F_i at pixels S.
        Perform gradient descent on E to update the pose P_i.
    end for
    Add Gaussians to G in unmapped areas of F_i.
    for N_map iterations do
        Choose a past frame F_k with pose P_k.
        Sample a subset S of pixels from F_k.
        Render the scene R using pose P_k, pixels S.
        Compute the error E between scene R and frame F_k at pixels S.
        Perform gradient descent on E to update each Gaussian in G.
    end for
end for
```

The loop over frames in Algorithm 1 is sequential, since each iteration depends on the pose and Gaussian map produced by previous iterations. We parallelize computation in scene rendering and the gradient computations on the pose and Gaussians.

First, rendering is data-parallel. Each Gaussian is projected onto the image independently, and each sampled pixel is computed independently by blending the Gaussians that cover it. Under the pixel-based parallelization scheme, we will assign each sampled pixel a warp whose threads split that pixel's Gaussians. The challenges are load imbalance, since pixels are covered by very different numbers of Gaussians, and warp divergence, since nearly transparent Gaussians are skipped.

Secondly, gradient computation is parallel across pixel–Gaussian pairs, since each sampled pixel computes a gradient for every Gaussian that contributed to it. However, many pixels contribute to the same Gaussian, so their gradients must be summed using reductions or atomic operations, which serialize when many threads update the same value. This is most extreme in tracking, where every sampled pixel contributes to the same camera pose. We will apply the same pixel-based scheme to gradient computation and reduce contention by combining gradients within a warp before updating shared values.

## 3 Challenge

Gaussian splatting SLAM is challenging to parallelize efficiently because its iterations are sequential and its parallelism is irregular. Each tracking and mapping iteration depends on the result of the previous one, so all parallelism must come from within a single rendering and gradient pass. Within a pixel, Gaussians must be blended in front-to-back depth order, which requires sorting and makes blending a sequential running product unless it is restructured as a parallel scan.

The work within each pass is irregular and divergent. The number of Gaussians covering each pixel varies with scene geometry and viewpoint, causing load imbalance. Within the blending loop, threads skip Gaussians that are nearly transparent at their pixel and stop early once their pixel becomes opaque, so threads in the same warp take different paths. Gradient computation adds communication between

threads, since many pixels contribute gradients to the same Gaussian and every sampled pixel in tracking contributes to the same camera pose. These updates require reductions or atomic operations, giving the gradient pass a high communication-to-computation ratio.

The workload also has low arithmetic intensity, so performance depends heavily on memory bandwidth and locality. The existing tile-based renderer achieves locality by loading each Gaussian into shared memory once and reusing it across all pixels in a 16×16 tile. With sparse sampling, however, only a few pixels per tile are rendered, so most threads sit idle. On the A100, warps execute 32 threads in lockstep, so divergence and idle threads directly waste compute, atomic operations to the same address are serialized, and limited registers and shared memory restrict how many warps can run at once. Sparse kernels may also be too small to fill the GPU, making launch overhead significant.

Through this project, we hope to learn how to parallelize a workload whose computation becomes sparse and irregular. In particular, we want to understand the trade-off between the tile-based scheme, which reuses data efficiently but leaves threads idle under sampling, and the pixel-based scheme, which keeps threads busy but gives up that reuse. We also hope to separate how much speedup comes from reducing work through sampling versus parallelizing it more efficiently, and to identify which factors ultimately limit how close Gaussian splatting SLAM can come to real-time performance.

## 4 Resources

Our project builds on SplaTAM [2], a Gaussian splatting SLAM system developed at CMU, which we will use as our reference implementation and starting codebase. The codebase includes a CUDA rasterizer for rendering and gradient computation, which serves as our tile-based baseline. We also use the original 3D Gaussian Splatting paper [3] as a reference for the rendering algorithm.

We will run all experiments on NVIDIA A100 GPUs on NCSA's Delta cluster. We will develop using single-GPU jobs and run final benchmarks with exclusive node access. SplaTAM was tested with PyTorch 1.12.1 and CUDA 11.6, which support the A100. We will use NVIDIA Nsight Systems and Nsight Compute to profile our kernels. Our implementation will use CUDA/C++ for performance-critical kernels and Python with PyTorch for the surrounding SLAM loop, error computation, parameter updates, dataset processing, and evaluation.

We will use the Replica dataset [4] as our primary benchmark, since it provides synthetic RGB-D sequences with exact depth and ground-truth camera poses for measuring tracking accuracy. We will also use the TUM-RGBD dataset [5], which contains real sensor data with noisier depth. As an optional demo, we may capture our own scenes with an iPhone Pro, which has a lidar depth sensor.

## 5 Goals and Deliverables

Our central goal is to determine how much faster Gaussian splatting SLAM can run through GPU-level optimization alone, without sacrificing accuracy. Our goals build on one another. We first establish a measured baseline, then reduce the amount of work through sparse pixel sampling, and finally restructure the GPU kernels so this sparser workload uses the hardware efficiently. Because Splatonic [1] reports results for similar optimizations on a different GPU, we base our performance targets on its numbers but set them conservatively, and we will revise them once we profile our own baseline. Throughout, we treat accuracy as a constraint, since a faster system is only useful if it still tracks the camera and reconstructs the scene correctly.

**Plan to achieve.**

- **Baseline.** Run SplaTAM on Delta A100s, reproduce its tracking accuracy on Replica, and profile the time spent in rendering and gradient computation, which Splatonic reports as 94.7% of runtime.
- **Sparse pixel sampling.** Pass sampled pixels into the existing tile-based kernels. We expect a 2–4× speedup in rendering and gradient computation, below Splatonic's reported 4.2× and 5.2× due to differences in hardware and implementation.
- **Pixel-based kernels.** Implement rendering and gradient kernels that assign each sampled pixel a warp, with early transparency checks and warp-level combining of gradients. We expect at least a further 2× speedup, since the tile-based kernels leave most threads idle under sampling.
- **End-to-end speedup.** Achieve a 5× end-to-end speedup over SplaTAM on the A100, with tracking error within 10% of the baseline and rendering quality within 1 dB PSNR. This is conservative relative to Splatonic's 14.6× because we keep SplaTAM's Python loop, whose overhead becomes significant as the kernels get faster.

**Hope to achieve.**

- **Merged rendering passes.** Combine SplaTAM's two rendering passes per iteration, one for color and one for depth and silhouette, into a single pass.
- **Concurrent tracking and mapping.** Overlap tracking of the next frame with mapping of the current one on separate CUDA streams.
- **10× end-to-end speedup.** At SplaTAM's roughly 0.4 frames per second, this would reach about 4 frames per second. We will restate this target once we measure our A100 baseline. Real-time performance at 30 frames per second would require nearly 75×, which we do not expect without algorithmic changes.

**If work goes slowly.** If the pixel-based gradient kernel is not working, we will pair the pixel-based rendering kernel with the existing gradient kernel. If the pixel-based scheme does not outperform the baseline, we will optimize the tile-based kernels for sparse sampling and analyze why the pixel-based scheme falls short.

**Evaluation.** We will measure speed and accuracy after each optimization on Replica and TUM-RGBD. Our analysis will determine how much of the speedup comes from reducing work through sampling versus parallelizing more efficiently, whether better thread utilization in the pixel-based scheme outweighs its loss of shared-memory reuse, which factor ultimately limits performance among divergence, memory bandwidth, atomic contention, and launch overhead, and how much accuracy sparse sampling costs.

**Deliverables and demo.** We will deliver our source code, profiling data, per-optimization speed and accuracy results, and a final report. At the poster session, we plan to show the baseline and our optimized version reconstructing a Replica scene side by side with frame rates displayed, and if time permits, a scene captured with an iPhone's lidar sensor.

## 6 Platform Choice

We chose NVIDIA A100 GPUs on NCSA's Delta cluster, programming in CUDA C++ for our kernels and Python with PyTorch for the surrounding SLAM loop. Our workload is massively data-parallel.

Each rendering and gradient pass involves hundreds of thousands to millions of Gaussians and thousands of sampled pixels, and the computation for each Gaussian or pixel is largely independent. This makes it a natural fit for a GPU, which can run tens of thousands of threads at once and has the high memory bandwidth needed to stream Gaussian data every iteration. A multi-core CPU would not expose enough parallelism to process this many independent computations quickly.

CUDA gives us the low-level control our optimizations require. Our pixel-based scheme depends on assigning work to individual warps, combining values across a warp's threads using warp shuffle instructions, and controlling how threads update shared gradients with atomic operations. These features are not available through higher-level frameworks. We keep the SLAM loop in Python because SplaTAM, our reference implementation, is written this way, which lets us replace its CUDA rasterizer with our own kernels while reusing the rest of the system. The A100 is also well matched to our reference code, since SplaTAM was tested with software versions that support it.

## 7 Timeline

| **Dates** | **Planned Work** |
|---|---|
| Nov. 9–15 | • Set up SplaTAM on Delta A100s and download the Replica and TUM-RGBD datasets.<br>• Reproduce SplaTAM's runtime and tracking accuracy on Replica.<br>• Profile rendering and gradient computation with Nsight. |
| Nov. 16–22 | • Implement sparse pixel sampling within the existing tile-based rasterizer, using random per-tile sampling for tracking and sampling of unmapped and textured regions for mapping.<br>• Measure its effect on speed and accuracy, and measure thread utilization to confirm idle threads.<br>• Begin the pixel-based rendering kernel. |
| Nov. 23 | • **Milestone report due.**<br>• Report baseline profiling results, sparse sampling results, and progress on the pixel-based kernel. |
| Nov. 24–29 | • Complete the pixel-based rendering kernel and verify that its output matches the baseline.<br>• Begin the pixel-based gradient kernel. |
| Nov. 30–Dec. 6 | • Complete the pixel-based gradient kernel and verify its gradients against the baseline.<br>• Add early transparency checks and warp-level combining of gradients.<br>• Integrate the new kernels into both tracking and mapping. |
| Dec. 7–10 | • Measure speed and accuracy after each optimization on Replica and TUM-RGBD.<br>• Generate figures and complete the final report. |
| Dec. 11 | • Present the final project poster and demo. |

## References


[1] Xiaotong Huang, He Zhu, Tianrui Ma, Yuxiang Xiong, Fangxin Liu, Zhezhi He, Yiming Gan, Zihan Liu, Jingwen Leng, Yu Feng, et al. SPLATONIC: Architectural support for 3D Gaussian splatting SLAM via sparse processing. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1–14. IEEE, 2026.

[2] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21357–21366. IEEE, 2024.

[3] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):139:1–139:14, 2023.

[4] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, et al. The Replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019.

[5] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 573–580. IEEE, 2012.